

# Lecture 2: Dimensionality Reduction

Master Course — Statistical Learning and Applications

Maximilien Drevet

## Abstract

TODO

## Contents

|           |                                                               |           |
|-----------|---------------------------------------------------------------|-----------|
| <b>1</b>  | <b>Motivation and Problem Formulation</b>                     | <b>3</b>  |
| 1.1       | Why Reduce Dimensionality? . . . . .                          | 3         |
| 1.2       | Problem Formulation . . . . .                                 | 3         |
| 1.3       | Linear vs. Nonlinear Methods . . . . .                        | 5         |
| <b>I</b>  | <b>Linear Methods</b>                                         | <b>5</b>  |
| <b>2</b>  | <b>Principal Component Analysis (PCA)</b>                     | <b>5</b>  |
| 2.1       | Derivation . . . . .                                          | 5         |
| 2.2       | Algorithm . . . . .                                           | 6         |
| 2.3       | Link with the SVD . . . . .                                   | 6         |
| 2.4       | Choosing the number of components . . . . .                   | 7         |
| 2.5       | Limitations of PCA . . . . .                                  | 8         |
| <b>3</b>  | <b>Random projections and the Johnson–Lindenstrauss lemma</b> | <b>9</b>  |
| 3.1       | The Johnson–Lindenstrauss lemma . . . . .                     | 9         |
| 3.2       | Random projection matrices . . . . .                          | 10        |
| 3.3       | Proof sketch (Gaussian case) . . . . .                        | 10        |
| 3.4       | Comparison with PCA . . . . .                                 | 10        |
| 3.5       | Extensions and applications . . . . .                         | 11        |
| <b>II</b> | <b>Nonlinear Methods</b>                                      | <b>12</b> |
| <b>4</b>  | <b>Neighbourhood-based embedding: t-SNE and UMAP</b>          | <b>12</b> |
| 4.1       | t-SNE . . . . .                                               | 12        |
| 4.1.1     | Motivation . . . . .                                          | 12        |
| 4.1.2     | High-dimensional affinities . . . . .                         | 12        |
| 4.1.3     | Low-dimensional affinities and the crowding problem . . . . . | 13        |
| 4.1.4     | Objective function . . . . .                                  | 14        |
| 4.1.5     | Practical considerations and pitfalls . . . . .               | 14        |
| 4.2       | UMAP . . . . .                                                | 15        |
| 4.2.1     | Motivation . . . . .                                          | 15        |
| 4.2.2     | High-dimensional graph . . . . .                              | 15        |
| 4.2.3     | Low-dimensional affinities . . . . .                          | 16        |
| 4.2.4     | Objective function . . . . .                                  | 17        |

|       |                                        |    |
|-------|----------------------------------------|----|
| 4.2.5 | Hyperparameters . . . . .              | 17 |
| 4.3   | Comparison of t-SNE and UMAP . . . . . | 17 |
| 4.4   | Common warnings . . . . .              | 18 |

# 1 Motivation and Problem Formulation

## 1.1 Why Reduce Dimensionality?

Modern datasets routinely live in high-dimensional spaces: an image of  $p \times p$  pixels yields a vector in  $\mathbb{R}^{p^2}$ ; a text document represented by word counts may have  $d = 10^5$  features; gene expression data can involve tens of thousands of variables measured on a handful of patients. Working directly in such high-dimensional spaces raises several difficulties.

- (i) **Curse of dimensionality.** As  $d$  grows, data points become increasingly spread out: distances concentrate, volumes explode, and the amount of data needed to maintain statistical accuracy grows exponentially. More precisely, for  $n$  points drawn uniformly in  $[0, 1]^d$ , the expected distance to the nearest neighbour scales as

$$\mathbb{E}[\min_{j \neq i} \|X_i - X_j\|] \sim c_d n^{-1/d},$$

which converges to the diameter of the cube as  $d \rightarrow \infty$  for fixed  $n$ . In other words, *all points look equally far from each other*, which cripples distance-based methods (clustering,  $k$ -NN, kernel methods).

- (ii) **Computational cost.** Many algorithms have complexity that depends on  $d$ : storing a covariance matrix costs  $O(d^2)$ , inverting it  $O(d^3)$ , and even a single pairwise distance computation is  $O(d)$ . Reducing the ambient dimension directly accelerates downstream tasks.
- (iii) **Noise and redundancy.** Not all coordinates carry useful information. Correlated features inflate the dimension without adding signal. Projecting onto a lower-dimensional subspace can *denoise* the data by discarding directions of small variance (which are often dominated by noise).
- (iv) **Visualisation.** Humans perceive at most three spatial dimensions. Any exploratory analysis of high-dimensional data requires a projection to  $\mathbb{R}^2$  or  $\mathbb{R}^3$ , raising the question: which projection best reveals the structure of the data?

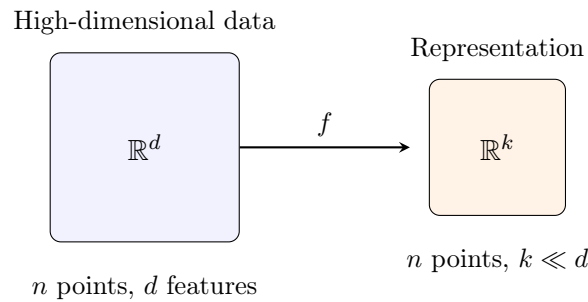


Figure 1: Dimensionality reduction seeks a map  $f : \mathbb{R}^d \rightarrow \mathbb{R}^k$  that preserves relevant structure while  $k \ll d$ .

## 1.2 Problem Formulation

Given a dataset  $X = \{x_1, \dots, x_n\} \subset \mathbb{R}^d$ , the goal is to find a low-dimensional representation  $Z = \{z_1, \dots, z_n\} \subset \mathbb{R}^k$  with  $k \ll d$  that preserves the “essential structure” of the data. The meaning of *essential structure* leads to two broad families of methods.

**Viewpoint 1: Preserve variance (or reconstruction).** Let  $\bar{x} = \frac{1}{n} \sum_{i=1}^n x_i$  denote the sample mean. If  $W = \text{span}(w_1, \dots, w_k)$  with  $w_1, \dots, w_k \in \mathbb{R}^d$  orthonormal, the orthogonal projection onto  $W$  is

$$\pi_W(x) = \sum_{j=1}^k \langle x, w_j \rangle w_j = W_k W_k^\top x,$$

where  $W_k = [w_1 \mid \dots \mid w_k] \in \mathbb{R}^{d \times k}$ . The low-dimensional representation of  $x$  is then the vector of coordinates  $z = W_k^\top x \in \mathbb{R}^k$ .

The goal is to find a  $k$ -dimensional linear subspace  $W$  that minimises the mean squared reconstruction error around the mean:

$$\min_{\substack{W \subset \mathbb{R}^d \\ \dim W = k}} \frac{1}{n} \sum_{i=1}^n \|(x_i - \bar{x}) - \pi_W(x_i - \bar{x})\|^2.$$

We center each point around  $\bar{x}$  because we care about the *spread* of the data, not its location: without centering, the leading direction would partly capture the mean rather than the direction of maximal variability. By Pythagoras,  $\|x_i - \bar{x}\|^2 = \|\pi_W(x_i - \bar{x})\|^2 + \|(x_i - \bar{x}) - \pi_W(x_i - \bar{x})\|^2$ , so minimising the reconstruction error is equivalent to maximising the *projected variance*:

$$\max_{\substack{W \subset \mathbb{R}^d \\ \dim W = k}} \frac{1}{n} \sum_{i=1}^n \|\pi_W(x_i - \bar{x})\|^2.$$

**Intuition.** The reconstruction error measures how much information is lost when we replace each (centered) point by its projection. Minimising the total error thus finds the subspace that best summarises the data in the least-squares sense. This is the starting point for **PCA** (Section 2).

**Remark 1.1.** Recall that the *sample variance* of the original data is:  $\text{Var}(X) = \frac{1}{n} \sum_i \|x_i - \bar{x}\|^2$ . This measures how spread out the data is around its mean. When we project onto a subspace  $W$ , each centered point  $\tilde{x}_i = x_i - \bar{x}$  becomes  $\pi_W(\tilde{x}_i)$ . A key observation is that the projected points are automatically centered:

$$\frac{1}{n} \sum_i \pi_W(\tilde{x}_i) = \pi_W \left( \frac{1}{n} \sum_i \tilde{x}_i \right) = 0,$$

where we used the linearity of  $\pi_W$  and the fact that centered data sums to zero. So the mean of the projected data is zero, and therefore the variance of the projected data is  $\frac{1}{n} \sum_i \|\pi_W(\tilde{x}_i) - 0\|^2$ , which is exactly the quantity we are maximising. That's why it's called the projected variance: it is literally the sample variance of the data after projection onto  $W$ .

**Viewpoint 2: Preserve distances or neighbourhoods.** Find an embedding  $z_1, \dots, z_n \in \mathbb{R}^k$  such that pairwise relationships are approximately conserved:

$$\|z_i - z_j\| \approx \delta(x_i, x_j) \quad \forall i, j,$$

where  $\delta$  can be Euclidean distance (leading to **Multidimensional Scaling (MDS)**, which finds coordinates that best reproduce a given distance matrix), geodesic distance (leading to **Isomap**), or a neighbourhood-based similarity (leading to **t-SNE**, **UMAP**, **Laplacian Eigenmaps**).

These two viewpoints are not mutually exclusive. In Section ?? we will see that *classical MDS*, a method that works from pairwise distances alone, recovers exactly the PCA solution when those distances are Euclidean. However, the distinction becomes important when the data lies on a curved manifold: linear methods like PCA may fail to capture nonlinear structure that neighbourhood-based methods can reveal.

**Remark 1.2** (The manifold hypothesis). A common assumption in machine learning is that high-dimensional data concentrates near a low-dimensional manifold  $\mathcal{M} \subset \mathbb{R}^d$  of intrinsic dimension  $k \ll d$ . For example, the set of natural images of a face (under varying pose and lighting) forms a manifold of dimension roughly equal to the number of degrees of freedom (pose angles, light direction), which is much smaller than the number of pixels. Dimensionality reduction can be seen as the task of recovering coordinates on  $\mathcal{M}$ .

### 1.3 Linear vs. Nonlinear Methods

A useful classification of dimensionality reduction techniques:

| Method              | Linear? | Preserves                    |
|---------------------|---------|------------------------------|
| PCA                 | Yes     | Global variance              |
| Random Projections  | Yes     | Pairwise distances (approx.) |
| Kernel PCA          | No      | Variance in feature space    |
| Laplacian Eigenmaps | No      | Local neighbourhood graph    |
| t-SNE / UMAP        | No      | Local neighbourhoods         |

Linear methods are computationally cheap, well-understood, and produce an explicit projection matrix. Nonlinear methods can capture curved structures but are typically more expensive and harder to interpret.

The rest of this lecture explores these methods in detail, starting with PCA and building towards nonlinear techniques.

## Part I

# Linear Methods

## 2 Principal Component Analysis (PCA)

### 2.1 Derivation

After centering the data, denote  $\tilde{x}_i = x_i - \bar{x}$  and form the data matrix  $\tilde{X} = [\tilde{x}_1 \mid \cdots \mid \tilde{x}_n] \in \mathbb{R}^{d \times n}$ . The (empirical) covariance matrix is

$$S = \frac{1}{n} \sum_{i=1}^n \tilde{x}_i \tilde{x}_i^\top = \frac{1}{n} \tilde{X} \tilde{X}^\top \in \mathbb{R}^{d \times d}.$$

$S$  is symmetric positive semi-definite, so it admits an eigendecomposition

$$S = \sum_{j=1}^d \lambda_j u_j u_j^\top, \quad \lambda_1 \geq \lambda_2 \geq \cdots \geq \lambda_d \geq 0,$$

where  $(u_1, \dots, u_d)$  is an orthonormal basis of eigenvectors.

**Proposition 2.1.** *The solution to the variance-maximisation (equivalently, reconstruction-error minimisation) problem of Viewpoint 1 is  $W^* = \text{span}(u_1, \dots, u_k)$ , i.e. the subspace spanned by the  $k$  leading eigenvectors of  $S$ . Moreover,*

$$\frac{1}{n} \sum_{i=1}^n \|\pi_{W^*}(\tilde{x}_i)\|^2 = \sum_{j=1}^k \lambda_j.$$

*Proof.* For any subspace  $W = \text{span}(w_1, \dots, w_k)$  with orthonormal  $w_j$ ,

$$\frac{1}{n} \sum_{i=1}^n \|\pi_W(\tilde{x}_i)\|^2 = \frac{1}{n} \sum_{i=1}^n \sum_{j=1}^k \langle \tilde{x}_i, w_j \rangle^2 = \sum_{j=1}^k w_j^\top S w_j.$$

We therefore need to solve

$$\max_{\substack{w_1, \dots, w_k \in \mathbb{R}^d \\ w_i^\top w_j = \delta_{ij}}} \sum_{j=1}^k w_j^\top S w_j.$$

By the Courant–Fischer (min-max) theorem, the maximum of  $\sum_{j=1}^k w_j^\top S w_j$  over orthonormal systems is  $\sum_{j=1}^k \lambda_j$ , attained when  $w_j = u_j$ .  $\square$

The vectors  $u_1, \dots, u_k$  are called the **principal components** (or principal directions). The coordinates  $z_i = W_k^\top \tilde{x}_i \in \mathbb{R}^k$  are the **principal scores** of  $x_i$ .

## 2.2 Algorithm

1. **Center** the data:  $\tilde{x}_i \leftarrow x_i - \bar{x}$ .
2. **Compute the covariance matrix**  $S = \frac{1}{n} \tilde{X} \tilde{X}^\top$ .
3. **Eigen-decompose**  $S$ : obtain eigenvalues  $\lambda_1 \geq \dots \geq \lambda_d$  and eigenvectors  $u_1, \dots, u_d$ .
4. **Truncate**: keep the first  $k$  eigenvectors  $W_k = [u_1 \mid \dots \mid u_k]$ .
5. **Project**:  $z_i = W_k^\top \tilde{x}_i \in \mathbb{R}^k$ .

**Complexity.** Step 2 costs  $O(d^2 n)$  and step 3 costs  $O(d^3)$ . When  $d \gg n$  it is cheaper to work with the *dual* Gram matrix  $G = \frac{1}{n} \tilde{X}^\top \tilde{X} \in \mathbb{R}^{n \times n}$  (see the kernel trick below). When only the first few components are needed, iterative methods (Lanczos, randomised SVD) reduce the cost further.

## 2.3 Link with the SVD

Write the centred data matrix

$$\tilde{X} = U \Sigma V^\top, \quad U \in \mathbb{R}^{d \times r}, \Sigma = \text{diag}(\sigma_1, \dots, \sigma_r), V \in \mathbb{R}^{n \times r},$$

where  $r = \text{rank}(\tilde{X})$  and  $\sigma_1 \geq \dots \geq \sigma_r > 0$ . The sample covariance matrix satisfies

$$S = \frac{1}{n} \tilde{X} \tilde{X}^\top = U \frac{\Sigma^2}{n} U^\top,$$

so the left singular vectors  $u_1, \dots, u_r$  of  $\tilde{X}$  are exactly the eigenvectors of  $S$ , with eigenvalues  $\lambda_j = \sigma_j^2/n$ . In particular, the PCA projection onto the top  $k$  components is

$$\tilde{X}_k = U_k U_k^\top \tilde{X},$$

where  $U_k = [u_1 \mid \dots \mid u_k]$ . Equivalently,

$$\tilde{X}_k = U_k \Sigma_k V_k^\top,$$

the SVD truncated to its  $k$  largest singular values.

**Eckart–Young–Mirsky theorem.** This truncation is optimal: among all matrices of rank at most  $k$ ,

$$\tilde{X}_k = \arg \min_{\text{rank}(M) \leq k} \|\tilde{X} - M\|_F^2 = \arg \min_{\text{rank}(M) \leq k} \|\tilde{X} - M\|_{\text{op}},$$

with

$$\|\tilde{X} - \tilde{X}_k\|_F^2 = \sum_{j=k+1}^r \sigma_j^2, \quad \|\tilde{X} - \tilde{X}_k\|_{\text{op}} = \sigma_{k+1}.$$

In words: PCA gives the best rank- $k$  approximation of the data matrix, in both the Frobenius and operator norms. This provides a second justification for PCA (beyond variance maximisation) as the projection that minimises the total squared reconstruction error  $\sum_{i=1}^n \|x_i - \hat{x}_i\|^2$ .

## 2.4 Choosing the number of components

The Eckart–Young–Mirsky theorem tells us that the squared reconstruction error of PCA with  $k$  components is  $\sum_{j=k+1}^r \sigma_j^2 = n \sum_{j=k+1}^r \lambda_j$ . Normalising by the total variance gives the *proportion of variance explained* by the first  $k$  components:

$$\text{PVE}(k) = \frac{\sum_{j=1}^k \lambda_j}{\sum_{j=1}^r \lambda_j} = 1 - \frac{\|\tilde{X} - \tilde{X}_k\|_F^2}{\|\tilde{X}\|_F^2}.$$

This quantity ranges from 0 (no component retained) to 1 (full reconstruction), and its complement is precisely the relative Frobenius error of the rank- $k$  approximation.

**Remark 2.2** (Why eigenvalues measure variance). Recall that the data is centred, so  $\bar{x} = 0$ . The empirical variance of the  $j$ -th coordinate across all data points is

$$\text{Var}(X^{(j)}) = \frac{1}{n} \sum_{i=1}^n \left( \tilde{x}_i^{(j)} \right)^2 = S_{jj},$$

i.e. the  $j$ -th diagonal entry of the sample covariance matrix  $S = \frac{1}{n} \tilde{X} \tilde{X}^\top$ . The *total variance* of the data is therefore

$$\sum_{j=1}^d \text{Var}(X^{(j)}) = \sum_{j=1}^d S_{jj} = \text{tr}(S) = \sum_{j=1}^r \lambda_j,$$

where the last equality is the standard linear algebra fact that the trace of a matrix equals the sum of its eigenvalues.

Now, the variance of the data projected onto the  $j$ -th principal direction  $u_j$  is

$$\text{Var}(u_j^\top \tilde{x}) = u_j^\top S u_j = \lambda_j,$$

where the first equality comes from the identity  $\text{Var}(u^\top \tilde{x}) = u^\top \text{Var}(x) u$  valid for any vector  $u$ , and the second equality holds as  $u_j$  is a normalized eigenvector of  $S$ . As a result, each eigenvalue  $\lambda_j$  represents the variance captured by the corresponding principal component. Summing the first  $k$  eigenvalues gives the variance retained by the rank- $k$  approximation, and dividing by  $\text{tr}(S)$  yields the proportion of variance explained  $\text{PVE}(k)$  defined above.

**Scree plot.** Plotting  $\lambda_j$  against  $j$  (or equivalently  $\text{PVE}(k)$  against  $k$ ) often reveals an *elbow*: a value of  $k$  beyond which the eigenvalues decay sharply. One common heuristic is to choose the smallest  $k$  such that  $\text{PVE}(k)$  exceeds a prescribed threshold (e.g. 90% or 95%), or equivalently such that the relative reconstruction error  $\|\tilde{X} - \tilde{X}_k\|_F^2 / \|\tilde{X}\|_F^2$  falls below 5–10%.

**Limitations of these heuristics.** The scree plot and variance threshold depend on the spectrum of  $S$ , which can be misleading: in high dimensions, noise alone can produce slowly decaying eigenvalues (the Marchenko–Pastur phenomenon), making the elbow hard to locate. More principled approaches exist, such as

- **Kaiser's rule:** keep components with  $\lambda_j > \bar{\lambda} = \frac{1}{d} \sum_j \lambda_j$  (eigenvalue above average).
- **Cross-validation:** hold out part of the data, reconstruct it using  $k$  components, and choose  $k$  minimising the held-out reconstruction error.
- **Random matrix theory:** compare the eigenvalues with those expected from a null (pure noise) model (Marchenko–Pastur distribution).

But this goes beyond the scope of these notes.

## 2.5 Limitations of PCA

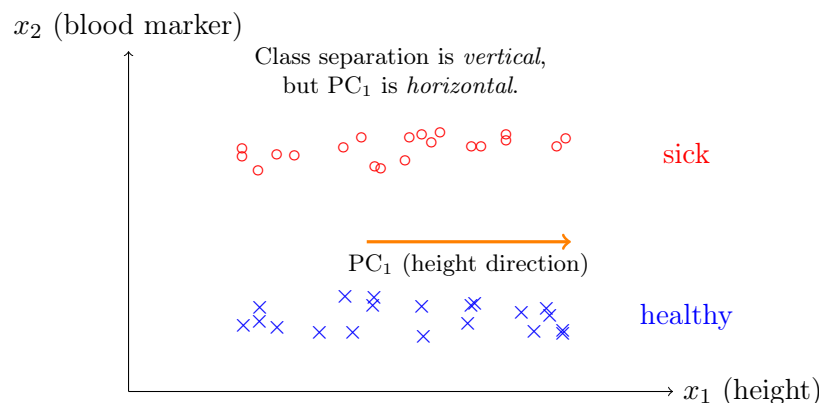
- **Linearity.** PCA finds a linear subspace; it cannot capture nonlinear structure (e.g. a Swiss roll).
- **Variance  $\neq$  relevance.** The directions of largest variance are not necessarily the most informative for a downstream task (e.g. classification).
- **Sensitivity to scaling.** Because PCA maximises variance, features with larger scales dominate. It is common practice to standardise each feature (zero mean, unit variance) before applying PCA, but this choice is problem-dependent.
- **Orthogonality constraint.** Principal components are mutually orthogonal, which may not align with the true latent structure.
- **Interpretability.** Each principal component is a linear combination of *all* original features, making interpretation difficult (sparse PCA addresses this).

### Example: Variance $\neq$ Relevance

Suppose we want to classify patients as **healthy** vs. **sick** using two measurements:

- $x_1$ : body height (in cm), ranging from 150 to 200,
- $x_2$ : a blood marker, ranging from 0 to 1.

Because height is measured in centimetres, its variance is orders of magnitude larger than that of the blood marker. PCA therefore selects the height direction as the first principal component: it captures most of the total variance. However, height has *nothing to do* with the disease. It is the blood marker  $x_2$  that separates the two classes:





If we keep only the first principal component, we project all points onto the horizontal axis and discard the only feature that distinguishes the two classes (the blood marker).

**Does standardisation fix this?** One might hope that standardising each feature to zero mean and unit variance solves the problem: height would no longer dominate simply because it is measured in larger units. Standardisation does remove this *scale artifact*, but the deeper issue remains.

**Example with a third feature.** Suppose we add a third measurement:

- $x_3$ : shoe size.

After standardisation every feature has variance 1, so no single feature dominates. However, height and shoe size are *strongly correlated* (tall people tend to have bigger feet), so they co-vary along a common direction. PCA on the standardised data finds:

| Component       | Interpretation                                                                      |
|-----------------|-------------------------------------------------------------------------------------|
| PC <sub>1</sub> | combination of height + shoe size (large variance, <i>irrelevant</i> for diagnosis) |
| PC <sub>2</sub> | blood marker (smaller variance, <b>relevant</b> for diagnosis)                      |

Even after standardisation, PCA still ranks the height/shoe-size direction first, because two correlated but *irrelevant* features create more variance than one *relevant* but uncorrelated feature.

The fundamental limitation is that *variance and relevance are different concepts*. Standardisation corrects for differences in measurement scale, but it cannot ensure that the high-variance directions are the informative ones for a downstream task. Supervised methods such as *Linear Discriminant Analysis* (LDA) address this by explicitly maximising *class separation* rather than total variance.

These limitations motivate the nonlinear methods covered in the following sections (kernel PCA, MDS, t-SNE, UMAP, autoencoders).

### 3 Random projections and the Johnson–Lindenstrauss lemma

PCA finds the *optimal* low-dimensional subspace for a given dataset, but computing it requires an eigendecomposition (or SVD) that can be expensive when  $d$  is very large. A striking result in high-dimensional geometry shows that, if one only cares about *approximately* preserving pairwise distances, a *random* projection works almost as well, with no dependence on the dimension  $d$ .

#### 3.1 The Johnson–Lindenstrauss lemma

**Theorem 3.1** (Johnson–Lindenstrauss, 1984). *Let  $\varepsilon \in (0, 1)$  and let  $x_1, \dots, x_n \in \mathbb{R}^d$ . There exists a linear map  $f : \mathbb{R}^d \rightarrow \mathbb{R}^k$  with*

$$k = O\left(\frac{\log n}{\varepsilon^2}\right)$$

*such that, for every pair  $i, j$ ,*

$$(1 - \varepsilon) \|x_i - x_j\|^2 \leq \|f(x_i) - f(x_j)\|^2 \leq (1 + \varepsilon) \|x_i - x_j\|^2.$$

The key surprise: the target dimension  $k$  depends only on  $\log n$  and  $\varepsilon$ , not on the ambient dimension  $d$ . For example,  $n = 10^6$  points can be embedded into  $k = O(\varepsilon^{-2} \log 10^6) \approx 14 \varepsilon^{-2}$  dimensions while distorting all pairwise distances by at most a factor  $(1 \pm \varepsilon)$ .

### 3.2 Random projection matrices

The map  $f$  can be constructed as  $f(x) = \frac{1}{\sqrt{k}} Ax$ , where  $A \in \mathbb{R}^{k \times d}$  is a random matrix with i.i.d. entries. Several distributions on the entries work:

1. **Gaussian:**  $A_{ij} \sim \mathcal{N}(0, 1)$ . This is the classical construction.
2. **Rademacher:**  $A_{ij} \in \{-1, +1\}$  uniformly at random. Equally effective, cheaper to generate.
3. **Sparse (Achlioptas):**  $A_{ij} \in \{-1, 0, +1\}$  with probabilities  $(\frac{1}{6}, \frac{2}{3}, \frac{1}{6})$ . The sparsity speeds up the matrix–vector product.

In each case the  $\frac{1}{\sqrt{k}}$  scaling ensures that  $\mathbb{E}[\|f(x)\|^2] = \|x\|^2$ , so that the projection is an *isometry in expectation*.

### 3.3 Proof sketch (Gaussian case)

Fix a single vector  $v \in \mathbb{R}^d$  with  $\|v\| = 1$  and let  $g_1, \dots, g_k$  denote the rows of  $A$ . Then

$$\|f(v)\|^2 = \frac{1}{k} \sum_{i=1}^k (g_i^\top v)^2.$$

Since  $g_i \sim \mathcal{N}(0, I_d)$ , each  $g_i^\top v \sim \mathcal{N}(0, 1)$ , so  $\|f(v)\|^2 = \frac{1}{k} \sum_{i=1}^k Z_i^2$  where  $Z_i \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1)$ . This is  $\frac{1}{k}$  times a  $\chi_k^2$  variable, whose concentration is controlled by the following sub-exponential tail bound:

$$\mathbb{P}\left[\left|\frac{1}{k} \sum_{i=1}^k Z_i^2 - 1\right| \geq \varepsilon\right] \leq 2 \exp\left(-\frac{k\varepsilon^2}{8}\right), \quad \varepsilon \in (0, 1).$$

Applying this to  $v = \frac{x_i - x_j}{\|x_i - x_j\|}$  for each of the  $\binom{n}{2}$  pairs and taking a union bound:

$$\mathbb{P}[\exists i < j : \|\|f(x_i) - f(x_j)\|^2 - \|x_i - x_j\|^2\| > \varepsilon \|x_i - x_j\|^2] \leq n^2 \exp\left(-\frac{k\varepsilon^2}{8}\right).$$

Choosing  $k \geq \frac{C \log n}{\varepsilon^2}$  for a suitable constant  $C$  makes this probability less than 1, so a good projection *exists* (and in fact is obtained with high probability by a single random draw).

### 3.4 Comparison with PCA

|                  | PCA                                        | Random projection                               |
|------------------|--------------------------------------------|-------------------------------------------------|
| Guarantee        | optimal variance / Eckart–Young            | $(1 \pm \varepsilon)$ distance preservation     |
| Target dimension | data-dependent                             | $O(\varepsilon^{-2} \log n)$ , data-independent |
| Computation      | SVD of $\tilde{X}$ : $O(\min(nd^2, n^2d))$ | matrix multiply: $O(ndk)$                       |
| Adaptivity       | exploits spectral structure                | oblivious to data                               |

When the spectrum of  $S$  decays quickly, PCA will achieve a much lower reconstruction error with the same  $k$ . Random projections are most useful when  $d$  is so large that computing the SVD is prohibitive, or when one needs a quick, data-oblivious dimensionality reduction with provable distance-preservation guarantees. In practice the two ideas can be combined: a random projection first reduces  $d$  to a manageable size, after which PCA extracts the dominant structure — an approach sometimes called *randomised PCA*.

### 3.5 Extensions and applications

- **Fast Johnson–Lindenstrauss transforms.** Ailon and Chazelle (2006) showed that premultiplying by a randomised Hadamard transform and then subsampling coordinates achieves the JL guarantee in time  $O(d \log d + k \log^2 n)$ , much faster than a dense matrix multiply.
- **Nearest-neighbour search.** Since random projections preserve distances, they can be used as a preprocessing step for approximate nearest-neighbour queries: project all points to  $\mathbb{R}^k$ , build a spatial index in the low-dimensional space, and search there.
- **Compressed sensing.** The JL lemma is closely related to the Restricted Isometry Property (RIP) used in compressed sensing: a random matrix satisfying JL for all sparse vectors also satisfies the RIP, enabling recovery of sparse signals from few linear measurements.
- **Streaming and distributed settings.** Because random projections are linear and data-oblivious, they compose naturally: partial projections computed on different machines (or at different times) can be combined, making them well suited to streaming and distributed data pipelines.

## Part II

# Nonlinear Methods

All methods in Part I share a common assumption: they seek a *linear* subspace onto which to project the data. When the data lies on or near a curved manifold embedded in  $\mathbb{R}^d$ , no linear subspace can faithfully capture its structure. We now relax this assumption and study methods that construct nonlinear low-dimensional representations.

## 4 Neighbourhood-based embedding: t-SNE and UMAP

We begin with the two most widely used visualisation techniques (t-SNE and UMAP) which aim to embed high-dimensional data into two or three dimensions while preserving *local* neighbourhood structure.

### 4.1 t-SNE

#### 4.1.1 Motivation

Suppose we have data  $x_1, \dots, x_n \in \mathbb{R}^d$  and wish to produce an embedding  $y_1, \dots, y_n \in \mathbb{R}^2$  (or  $\mathbb{R}^3$ ) for visualisation. We want nearby points in the original space to remain nearby in the embedding, and ideally for distinct clusters to be visually separated.

PCA, being a linear method, maps data onto the plane of greatest variance. But variance maximisation does not in general preserve local neighbourhoods: two points that are close on a curved manifold may be far apart when projected onto a linear subspace. *t-distributed Stochastic Neighbour Embedding* (t-SNE), introduced by [Van der Maaten and Hinton, 2008], takes a fundamentally different approach: it defines a probability distribution over pairs of points in both the high-dimensional and low-dimensional spaces, then minimises the mismatch between the two.

#### 4.1.2 High-dimensional affinities

The key idea of SNE is to represent the neighbourhood structure of each point  $x_i$  as a probability distribution. To do this, we place a Gaussian with variance  $\sigma_i^2$  centred at  $x_i$ . Points that fall in high-density regions of this Gaussian (i.e. close to  $x_i$ ) are considered likely neighbours; points in low-density regions (far from  $x_i$ ) are unlikely neighbours.

Formally, for each pair  $(i, j)$ , we define the conditional probability that  $x_i$  would pick  $x_j$  as its neighbour under this Gaussian:

$$p_{j|i} = \frac{\exp(-\|x_i - x_j\|^2 / 2\sigma_i^2)}{\sum_{k \neq i} \exp(-\|x_i - x_k\|^2 / 2\sigma_i^2)}, \quad p_{i|i} = 0. \quad (1)$$

The bandwidth  $\sigma_i$  is specific to each point: in dense regions of the data, a smaller  $\sigma_i$  is appropriate, while in sparse regions a larger value is needed. Rather than setting these manually, t-SNE determines each  $\sigma_i$  by fixing a user-chosen parameter called the *perplexity*, which can be interpreted as a smooth measure of the effective number of neighbours each point should have.

Formally, the perplexity of the conditional distribution  $P_i$  is defined as

$$\text{Perp}(P_i) = 2^{H(P_i)}, \quad \text{with} \quad H(P_i) = - \sum_{j \neq i} p_{j|i} \log_2 p_{j|i}, \quad (2)$$

where  $H(P_i)$  is the Shannon entropy (in bits) of the distribution over neighbours of  $x_i$ . For a given target perplexity (typical values range from 5 to 50), t-SNE performs a binary search over  $\sigma_i$  until  $\text{Perp}(P_i)$  matches the target. A perplexity of, say, 30 means that each point's

Gaussian is calibrated so that its neighbour distribution has an effective support of roughly 30 points.

**Remark 4.1** (Intuition: perplexity as an effective neighbour count.). Consider the extreme case where  $P_i$  is exactly uniform over  $k$  points and zero elsewhere. Then each of the  $k$  neighbours receives probability  $1/k$ , so

$$H(P_i) = - \sum_{j=1}^k \frac{1}{k} \log_2 \frac{1}{k} = \log_2 k \quad \implies \quad \text{Perp}(P_i) = 2^{\log_2 k} = k. \quad (3)$$

In this idealised setting the perplexity equals exactly the number of neighbours. More generally, entropy grows with the “spread” of  $P_i$  (the more points that receive non-negligible probability, the higher  $H(P_i)$ ) and the exponential in the perplexity definition converts this logarithmic quantity back into an interpretable count. Thus, setting a target perplexity of 30 is akin to asking: *calibrate each  $\sigma_i$  so that roughly 30 points contribute meaningfully to  $P_i$ .*

**Symmetrisation.** The conditional probabilities  $p_{j|i}$  defined in (1) are in general asymmetric: a point  $x_i$  in a dense cluster may assign high probability to a nearby point  $x_j$ , while  $x_j$ , sitting in a sparser region with a larger  $\sigma_j$ , spreads its probability mass more widely and assigns a lower value to  $x_i$ . To obtain a single symmetric distribution over pairs, t-SNE defines

$$p_{ij} = \frac{p_{j|i} + p_{i|j}}{2n}, \quad (4)$$

where  $n$  is the number of data points. This ensures  $p_{ij} = p_{ji}$  and  $\sum_{i,j} p_{ij} = 1$ . The averaging also guarantees that every point makes a minimum contribution to the cost function: even an outlier  $x_i$  satisfies  $\sum_j p_{ij} \geq 1/(2n)$ , which prevents it from being effectively ignored during optimisation.

#### 4.1.3 Low-dimensional affinities and the crowding problem

In the low-dimensional map, each high-dimensional point  $x_i$  has a counterpart  $y_i \in \mathbb{R}^d$  (typically  $d = 2$ ). A natural first choice made in the original *Stochastic Neighbour Embedding* (SNE) by [Hinton and Roweis, 2002] is to mirror the high-dimensional construction and use a Gaussian kernel:

$$q_{ij}^{\text{SNE}} = \frac{\exp(-\|y_i - y_j\|^2)}{\sum_{k \neq l} \exp(-\|y_k - y_l\|^2)}, \quad q_{ii}^{\text{SNE}} = 0. \quad (5)$$

**The crowding problem.** This symmetric choice, however, suffers from a fundamental geometric difficulty. In  $\mathbb{R}^D$  the volume of a thin spherical shell at radius  $r$  grows as  $r^{D-1}$ : when  $D$  is large, an enormous number of points can sit at moderate distances from any given point. In the low-dimensional map ( $d = 2$  or  $3$ ), the corresponding shell volume grows only as  $r^{d-1}$ , so there is far less room to place all those moderately distant neighbours. A Gaussian kernel, which decays quickly, forces the optimiser to squeeze these points into a small annulus around each centre, collapsing distinct groups into a featureless blob.

**Solution: a heavy-tailed kernel.** t-SNE resolves the crowding problem by replacing the Gaussian in the low-dimensional space with a Student- $t$  distribution with one degree of freedom (i.e. a Cauchy kernel):

$$q_{ij} = \frac{(1 + \|y_i - y_j\|^2)^{-1}}{\sum_{k \neq l} (1 + \|y_k - y_l\|^2)^{-1}}, \quad q_{ii} = 0. \quad (6)$$

The heavier tails of the Cauchy distribution mean that moderately similar points can sit *further apart* in the map without incurring a large cost, while genuinely close neighbours are still pulled

tightly together. In effect, the mismatch in tail weight between  $p_{ij}$  (Gaussian) and  $q_{ij}$  (Cauchy) creates an attractive force for nearby pairs and a repulsive force for distant ones, naturally carving out gaps between clusters.

#### 4.1.4 Objective function

t-SNE minimises the Kullback–Leibler divergence from  $P$  to  $Q$ :

$$\mathcal{L}(y_1, \dots, y_n) = \text{KL}(P\|Q) = \sum_{i \neq j} p_{ij} \log \frac{p_{ij}}{q_{ij}}.$$

Because KL divergence is asymmetric, this objective penalises configurations where  $q_{ij}$  is small but  $p_{ij}$  is large (nearby points mapped far apart) much more than the reverse (distant points mapped close together). The emphasis is therefore on **preserving local structure**: if two points are neighbours in  $\mathbb{R}^d$ , they should remain neighbours in the embedding.

The gradient takes a convenient form:

$$\frac{\partial \mathcal{L}}{\partial y_i} = 4 \sum_{j \neq i} (p_{ij} - q_{ij}) (1 + \|y_i - y_j\|^2)^{-1} (y_i - y_j).$$

Each term acts like a spring between  $y_i$  and  $y_j$ : if  $p_{ij} > q_{ij}$  the spring is attractive (the points are too far apart in the map), and if  $p_{ij} < q_{ij}$  it is repulsive. Optimisation is carried out by gradient descent, typically with momentum and an *early exaggeration* phase (multiplying all  $p_{ij}$  by a factor, e.g. 4, during the first few hundred iterations) to encourage clusters to separate early on.

**Remark 4.2** (A note on the asymmetry of KL divergence). Consider a single pair  $(i, j)$ :

- If  $p_{ij}$  is large (neighbours in  $\mathbb{R}^d$ ) and  $q_{ij}$  is small (far apart in the map), the term  $p_{ij} \log(p_{ij}/q_{ij})$  is large and positive, and this is heavily penalised.
- If  $p_{ij}$  is small (distant in  $\mathbb{R}^d$ ) and  $q_{ij}$  is large (close in the map), then  $p_{ij}$  multiplies the log, so the penalty is small.

Thus, tearing apart true neighbours is expensive; falsely placing distant points together is cheap. This is precisely why t-SNE is good at preserving local structure but indifferent to global layout. (Had we minimised  $\text{KL}(Q\|P)$  instead, the emphasis would be reversed.)

#### 4.1.5 Practical considerations and pitfalls

1. **Perplexity.** The perplexity hyperparameter controls the effective neighbourhood size. Low perplexity ( $\sim 5$ ) reveals fine local structure but can fragment clusters; high perplexity ( $\sim 50$ ) gives a more global view but may merge distinct groups. It is good practice to run t-SNE at several perplexity values and compare.
2. **Non-convexity and randomness.** The KL objective is non-convex, and the embedding is initialised randomly (or via PCA). Different runs can produce qualitatively different plots. Always run multiple restarts and check consistency.
3. **Distances in the map are not meaningful.** t-SNE preserves neighbourhood *ranks*, not distances. The absolute distance between two clusters in a t-SNE plot says nothing about their distance in the original space. Similarly, the relative sizes of clusters are not preserved.

4. **Global structure is not preserved.** The KL asymmetry means the algorithm has no incentive to faithfully represent large-scale inter-cluster relationships. Conclusions such as “cluster A is between clusters B and C” cannot be drawn from a t-SNE plot alone.
5. **Not a general-purpose preprocessor.** Because the map is non-parametric (there is no function  $f : \mathbb{R}^d \rightarrow \mathbb{R}^2$  that can be applied to new points), and because global geometry is lost, t-SNE embeddings should **not** be used as input features for downstream classification or clustering algorithms. Their proper use is *exploratory visualisation*.
6. **Complexity.** The naïve algorithm is  $O(n^2)$  per iteration (computing all pairwise  $q_{ij}$ ). The Barnes–Hut approximation reduces this to  $O(n \log n)$  by using a tree structure to approximate the repulsive forces, and the FIt-SNE implementation further accelerates computation using fast Fourier transforms.

## 4.2 UMAP

### 4.2.1 Motivation

*Uniform Manifold Approximation and Projection* (UMAP), introduced by [McInnes et al., 2018], addresses several practical limitations of t-SNE while retaining its strengths. Like t-SNE, UMAP produces low-dimensional embeddings that emphasise local structure, but it tends to better preserve aspects of global geometry, runs significantly faster on large datasets, and has a cleaner mathematical foundation rooted in topology.

We will not develop the full topological machinery but instead focus on the *algorithmic* description, which closely parallels t-SNE and makes the differences transparent.

### 4.2.2 High-dimensional graph

Where t-SNE converts pairwise distances into a full matrix of probabilities, UMAP builds a *weighted k-nearest-neighbour graph* whose edges encode local similarity. The construction proceeds in three steps.

**Step 1: local distances and connectivity.** For each data point  $x_i$ , UMAP first identifies its  $k$  nearest neighbours under a user-chosen metric  $d$  (Euclidean by default). It then records

$$\rho_i = \min_{j \in \text{kNN}(i)} d(x_i, x_j),$$

the distance from  $x_i$  to its *closest* neighbour. The role of  $\rho_i$  will become clear shortly: it enforces **local connectivity**, guaranteeing that no point is isolated in the graph.

**Step 2: local similarity function.** We need a function that converts the distance between two points into a similarity score: large when  $x_j$  is close to  $x_i$ , decaying smoothly to zero as  $x_j$  moves away. UMAP defines

$$w_{j|i} = \exp\left(-\frac{\max(0, d(x_i, x_j) - \rho_i)}{\sigma_i}\right), \quad (7)$$

where  $\sigma_i > 0$  is a per-point bandwidth (determined in Step 3). Once all bandwidths are fixed and the weights symmetrised (Step 4), these values will become the edge weights of the final graph.

**Remark 4.3.** As in t-SNE, a kernel converts distances into similarities: neighbours at distance much smaller than  $\sigma_i$  receive weight close to 1, while those much farther than  $\sigma_i$  receive weight close to 0. The bandwidth  $\sigma_i$  controls where this transition occurs.

**Remark 4.4** (Exponential kernel). The exponential kernel converts distances into similarities: neighbours at distance much smaller than  $\sigma_i$  receive weight close to 1, while those much farther than  $\sigma_i$  receive weight close to 0. This sharp-but-smooth cutoff is desirable when building a graph in high-dimensional space: local distances are statistically reliable, but distant pairwise distances become increasingly noisy and uninformative due to the curse of dimensionality, so we want them to contribute negligible weight. A heavy-tailed kernel such as  $(1 + d^2)^{-1}$  would let distant points “leak” influence into the graph; the exponential suppresses them effectively. (The choice also arises naturally from UMAP’s topological framework based on fuzzy simplicial sets, where the exponential is the canonical membership function associated with a locally normalised Riemannian metric.)

**Remark 4.5** (Subtraction of  $\rho_i$ ). By shifting all distances so that the nearest neighbour is at effective distance zero, UMAP ensures  $w_{j^*|i} = 1$  for the closest neighbour  $j^*$ . This enforces *local connectivity*: every point is connected to at least one other point with full weight, regardless of how sparse or dense its surroundings are. Without this shift, points in very sparse regions could end up with all edge weights near zero, effectively becoming isolated in the graph.

**Step 3: adaptive bandwidth.** The bandwidth  $\sigma_i$  must adapt to the local density around each point: in a dense region  $\sigma_i$  should be small (keeping the neighbourhood tight), while in a sparse region it should be large (reaching out to more distant points). UMAP determines  $\sigma_i$  by binary search on the constraint

$$\sum_{j \in \text{kNN}(i)} w_{j|i} = \sum_{j \in \text{kNN}(i)} \exp\left(-\frac{\max(0, d(x_i, x_j) - \rho_i)}{\sigma_i}\right) = \log_2 k. \quad (8)$$

The logic mirrors the perplexity condition of t-SNE (Equation 2): the sum of edge weights leaving  $x_i$  is a measure of how many *effective* neighbours it has. By fixing this sum to  $\log_2 k$ , UMAP ensures every point sees a comparable number of effective neighbours, compensating for variations in local density. The target value  $\log_2 k$  is chosen so that, in a perfectly uniform neighbourhood of  $k$  equidistant points, each contributes a weight of  $(\log_2 k)/k$ , a moderate fraction that keeps the graph neither too sparse nor too dense.

**Step 4: symmetrisation and graph construction.** Because  $\rho_i \neq \rho_j$  and  $\sigma_i \neq \sigma_j$  in general, the weights are asymmetric:  $w_{j|i} \neq w_{i|j}$ . To obtain an undirected graph, UMAP symmetrises with a *fuzzy union*:

$$w_{ij} = w_{j|i} + w_{i|j} - w_{j|i} w_{i|j}. \quad (9)$$

Reading each  $w$  as the probability of an edge existing, this is simply the inclusion–exclusion formula  $P(A \cup B) = P(A) + P(B) - P(A)P(B)$  for two independent events. The result is a symmetric weight  $w_{ij} \in [0, 1]$ .

Note the contrast with t-SNE, which symmetrises its conditional probabilities by *averaging* ( $p_{ij} = \frac{1}{2}(p_{j|i} + p_{i|j})$ ). The fuzzy-union rule is more aggressive: if *either* point considers the other a close neighbour, the edge receives a high weight. This tends to preserve more connections in the graph, particularly between clusters of different densities.

### 4.2.3 Low-dimensional affinities

In the embedding space, UMAP uses a smooth family of curves. The default choice is

$$v_{ij} = (1 + a \|y_i - y_j\|^{2b})^{-1},$$

where  $a$  and  $b$  are parameters fitted so that the curve approximates  $\exp(-(\|y_i - y_j\| - \text{min\_dist}))$  for  $\|y_i - y_j\| > \text{min\_dist}$  and is close to 1 for  $\|y_i - y_j\| \leq \text{min\_dist}$ . The hyperparameter `min_dist` controls how tightly points can cluster in the embedding.

**Connection with t-SNE.** When  $a = 1$  and  $b = 1$ , this reduces to  $(1 + \|y_i - y_j\|^2)^{-1}$ , which is exactly the Cauchy kernel of t-SNE. UMAP thus generalises the t-SNE low-dimensional kernel.



#### 4.2.4 Objective function

Rather than KL divergence, UMAP minimises the *cross-entropy* of the two weighted graphs:

$$\mathcal{L} = \sum_{i \neq j} \left[ w_{ij} \log \frac{w_{ij}}{v_{ij}} + (1 - w_{ij}) \log \frac{1 - w_{ij}}{1 - v_{ij}} \right].$$

This can be decomposed into an **attractive** term (first summand: pull embedded neighbours together) and a **repulsive** term (second summand: push non-neighbours apart). Compared with t-SNE’s KL divergence, the cross-entropy also penalises placing distant points too close, which encourages better preservation of global structure.

**Optimisation.** The UMAP loss decomposes into an **attractive** term (summing over pairs with large  $w_{ij}$ ) and a **repulsive** term (summing over pairs with small  $w_{ij}$ ). The attractive term is cheap:  $w_{ij}$  is nonzero only for the  $\Theta(nk)$  edges in the  $k$ -NN graph. The repulsive term, however, involves all  $O(n^2)$  non-neighbour pairs, making a naïve gradient computation infeasible for large datasets. However, UMAP sidesteps this with *stochastic gradient descent and negative sampling*, borrowing an idea from word-embedding methods (word2vec). At each step:

1. **Attractive step.** Sample an edge  $(i, j)$  from the graph with probability  $\propto w_{ij}$ . Compute the attractive gradient and move  $y_i, y_j$  closer.
2. **Repulsive step.** For the same anchor  $i$ , draw  $m$  points  $j'_1, \dots, j'_m$  uniformly at random (typically  $m = 5$ ). With high probability these are non-neighbours. Compute the repulsive gradient for each pair  $(i, j'_\ell)$  and push them apart.
3. Only the embeddings of the involved points are updated.

Each step touches  $O(m)$  points; a full pass over all  $\Theta(nk)$  edges therefore costs  $O(nkm)$ , which is linear in  $n$  since  $k$  and  $m$  are small constants. By contrast, Barnes–Hut t-SNE uses a tree-based approximation of the repulsive forces at  $O(n \log n)$  cost per iteration, which is asymptotically comparable but harder to implement and parallelise. The simplicity of negative sampling is one of the main reasons UMAP scales better in practice.

#### 4.2.5 Hyperparameters

1. **n\_neighbors** ( $k$ ). Controls the size of the local neighbourhood, analogous to perplexity in t-SNE. Small  $k$  emphasises very fine local structure; large  $k$  captures more of the global layout. Typical values: 10–50.
2. **min\_dist**. Controls the minimum distance between embedded points. Small values ( $\sim 0.0$ ) produce tight, separated clusters; larger values ( $\sim 0.5$ ) give a more uniform spread.
3. **n\_components**. Unlike t-SNE, which is almost exclusively used with 2 or 3 components, UMAP can produce useful embeddings in higher dimensions (e.g. 10–50) for use as a preprocessing step.
4. **Metric**. UMAP accepts arbitrary distance functions (Euclidean, cosine, Hamming, ...), making it applicable to a wider range of data types than t-SNE.

### 4.3 Comparison of t-SNE and UMAP

|                    | t-SNE                              | UMAP                                             |
|--------------------|------------------------------------|--------------------------------------------------|
| High-dim. affinity | Gaussian, adaptive $\sigma_i$      | Exponential with local connectivity ( $\rho_i$ ) |
| Low-dim. kernel    | Cauchy (Student- $t$ , $\nu = 1$ ) | Parametric family $(1 + a\ y\ ^{2b})^{-1}$       |
| Objective          | KL( $P\ Q$ )                       | Binary cross-entropy                             |
| Optimisation       | GD + momentum + early exaggeration | SGD + negative sampling                          |
| Global structure   | poorly preserved                   | better (repulsive CE term)                       |
| Scalability        | $O(n \log n)$ with Barnes-Hut      | $O(n \cdot k)$ per epoch, faster in practice     |
| Embedding dim.     | typically 2–3                      | 2–3 for visualisation, higher for preprocessing  |
| New points         | not supported natively             | <code>transform</code> method available          |

#### 4.4 Common warnings

Both t-SNE and UMAP are powerful visualisation tools, but they share fundamental limitations that users must keep in mind:

1. **Cluster sizes are meaningless.** Both algorithms use adaptive bandwidths ( $\sigma_i$  or  $\rho_i/\sigma_i$ ), so a dense cluster of 100 points and a diffuse cluster of 1000 points may appear the same size in the embedding.
2. **Inter-cluster distances are unreliable.** The distance between two clusters in the plot does not reflect their distance in the original space. Never conclude that “cluster A is closer to B than to C” from the embedding alone.
3. **Apparent structure can be an artefact.** With certain hyperparameter choices, both methods can split a single continuous distribution into spurious clusters, or merge genuinely distinct groups. Always vary hyperparameters (perplexity / `n_neighbors`, `min_dist`) and check robustness.
4. **Visualisation  $\neq$  analysis.** These methods are for *generating hypotheses*, not testing them. Any pattern observed in a t-SNE or UMAP plot should be confirmed by quantitative analysis (e.g. clustering metrics, statistical tests) in the original space.

# Python Exercises: Dimensionality Reduction

## Setup

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from sklearn.datasets import load_digits, make_swiss_roll, make_moons,
  make_blobs
4 from sklearn.decomposition import PCA
5 from sklearn.manifold import TSNE
6 from umap import UMAP
7 from time import time
```

## Exercise 1: Warm-up on synthetic 2D data

We start with data that is *already* low-dimensional to build intuition about what each method preserves and what it distorts.

**1.1 Generate three-cluster data.** Use `make_blobs` to create  $n = 600$  points in  $\mathbb{R}^2$  with 3 well-separated isotropic clusters (use `centers=3`, `cluster_std=0.5`). Plot them, coloured by label.

**1.2 Embed into 2D with PCA, t-SNE, and UMAP.** Since the data is already 2D, a faithful method should approximately recover the original layout.

Fit PCA with 2 components, t-SNE with default parameters (`perplexity=30`), and UMAP with default parameters (`n_neighbors=15`, `min_dist=0.1`).

Plot the three embeddings, and colour the points by the original labels.

### 1.3 Questions.

- Does PCA change anything? Why or why not?
- Do t-SNE and UMAP preserve the relative distances *between* clusters?
- Do they preserve the *shape* (spread, aspect ratio) of each cluster?

**1.4 Anisotropic clusters.** Regenerate the data but now use `cluster_std=[0.5, 2.0, 0.5]` so that one cluster is much more spread out than the others. Repeat the three embeddings.

*Question.* Does t-SNE preserve the fact that one cluster is larger? What about UMAP? What about PCA?

## Exercise 2: The Swiss Roll

The Swiss Roll is a classic example of data that lies on a 2D manifold embedded in 3D. A good nonlinear method should “unroll” it; a linear method cannot.

**2.1 Generate and visualise.** Use `make_swiss_roll` with  $n = 2000$ , `noise=0.3`. The function returns `X` (3D coordinates) and `t` (the position along the roll, useful for colouring).

Make a 3D scatter plot coloured by `t`.

*Hint.* Use `fig.add_subplot(111, projection='3d')` and use the documentation [https://scikit-learn.org/stable/auto\\_examples/manifold/plot\\_swissroll.html](https://scikit-learn.org/stable/auto_examples/manifold/plot_swissroll.html).

**2.2 PCA projection.** Apply PCA with 2 components. Plot the result coloured by `t`.

*Question.* Does PCA unroll the manifold? Why not?

**2.3 t-SNE embedding.** Apply t-SNE with `perplexity=30`. Plot and colour by `t`.

*Question.* Does it unroll the manifold? Is the colour gradient smooth and monotone, or fragmented?

**2.4 UMAP embedding.** Apply UMAP with `n_neighbors=15`, `min_dist=0.1`. Plot and colour by `t`.

*Question.* Compare with t-SNE. Which method better preserves the global ordering along the roll?

**2.5 Hyperparameter sensitivity.** Try t-SNE with perplexities in  $\{5, 15, 30, 50, 100\}$  and UMAP with `n_neighbors` in  $\{5, 15, 50, 100\}$ .

*Questions.*

- For t-SNE, at which perplexity (if any) does the roll unroll cleanly?
- For UMAP, how does increasing `n_neighbors` affect the result?
- What happens with very small values for both?

### Exercise 3: Handwritten digits

Now we move to real high-dimensional data: the digits dataset ( $n = 1797$ ,  $d = 64$ ).

**3.1 Load and inspect.** Load the digits dataset. Display a grid of example images (e.g.  $2 \times 5$ , one per digit class) using `imshow` with `reshape(8,8)`.

**3.2 PCA: variance and visualisation.** Fit PCA with all 64 components.

- (a) Plot the cumulative explained variance ratio. How many components are needed to capture 90% of the variance?
- (b) Plot the data projected onto the first two principal components, coloured by digit label.
- (c) *Question.* Are the digit classes well separated in 2 PCA dimensions? Which digits overlap most?

**3.3 t-SNE visualisation.** Apply t-SNE with `perplexity=30` to the **full 64-dimensional** data. Plot coloured by digit label.

*Hint.* t-SNE on 64 dimensions is fine for  $n \approx 1800$ , but for larger datasets a common trick is to first reduce to  $\sim 30$  PCA dimensions and then apply t-SNE.

*Question.* How does the separation compare to the PCA plot?

**3.4 UMAP visualisation.** Apply UMAP with default parameters. Plot coloured by digit label.

*Question.* Compare with t-SNE: are the clusters similarly shaped? Is one method faster?

**3.5 Trustworthiness.** A quantitative way to evaluate an embedding is *trustworthiness*: it measures whether the nearest neighbours in the low-dimensional embedding were also near in the original space.

```
1 from sklearn.manifold import trustworthiness
```

Compute trustworthiness (with  $k = 15$ ) for PCA, t-SNE, and UMAP.

*Question.* Which method has the highest trustworthiness? Is this consistent with what you see in the plots?

## References

- [Hinton and Roweis, 2002] Hinton, G. E. and Roweis, S. (2002). Stochastic neighbor embedding. *Advances in neural information processing systems*, 15.
- [McInnes et al., 2018] McInnes, L., Healy, J., Saul, N., and Großberger, L. (2018). Umap: Uniform manifold approximation and projection. *Journal of Open Source Software*, 3(29).
- [Van der Maaten and Hinton, 2008] Van der Maaten, L. and Hinton, G. (2008). Visualizing data using t-sne. *Journal of machine learning research*, 9(11).